

GUÍA COMPLETA • USUARIO +
TÉCNICO • V1.1

PalmerP

Guía Completa del Sistema

Fleet independiente + Triple Backup 3-2-1 + Updates 1-click como tú
Por qué cada pieza existe y cómo operarla paso a paso, sin humo y sin huecos

Fleet	3-2-1	1-Click	02:00
1 casa por cliente su Vercel-Supabase-R2-MiniPC	3 copias · 2 medios · 1 off-site 90d vault immutable	cliente actualiza como tú misma branch, mismo build	backup diario 02:00 + cron Vercel 02:30

■ La promesa

Cada cliente tiene **su casa** (no un piso compartido), **3 llaves** cada noche 02:00 y **la misma llave inglesa que tú** para actualizar.
Si Vercel cae, quedan 2 copias. Si pierdes la clave, el vault no se abre — a propósito.

TÚ Palm-ERP/palmerp template	→ template	CLIENTE-X fork privado push main → Vercel	→	3 DESTINOS 02:00 A 7d B 30d C 90d
publicas mejora		Supabase pooler 6543 + MiniPC siempre ON		vault immutable solo tu clave abre

Para quién: **dueño** (cap. 4 y 9, sin código) + **instalador** (cap. 5, comandos literales) + **tú** (fleet). Cada capítulo es independiente: ve al que necesites.

Índice — 13 capítulos

1	La idea en 2 minutos — por qué este sistema	4
2	Mapa visual — cómo encaja todo	4
3	Por qué cada decisión (sin humo)	5
4	Manual de Usuario — lo que ves cada día	6
5	Manual del Instalador — de 0 a 100%	7
6	Triple Backup 3-2-1 — tus 3 seguros	10
7	MiniPC — copia A y cerebro local	11
8	Actualizar como yo — 1 click sin miedo	12
9	Operación diaria — 1 minuto	13
10	Seguridad y claves	14
11	Troubleshooting	14
12	Checklists y entrega	15
13	Glosario y referencias	16

1 La idea en 2 minutos — por qué este sistema

Antes: un SaaS central (*.palmerp.es + 1 Supabase) es barato pero te ata. Si Vercel cae, caen todos. Si un cliente quiere irse, no le puedes dar su base limpiamente. Si quieres ponerle un modelo local que aprende de su negocio, mezclarías datos. Y la factura de Vercel la pagas tú por todos.

Ahora: **fleet independiente** — cada cliente su GitHub (fork template), su Vercel, su Supabase, su **R2 Cloudflare 10GB gratis** y su MiniPC. Tú mantienes **Palm-ERP/palmerp** como upstream; cada **git push origin main** suyo despliega solo su ERP. Él paga su nube, tú pagas solo el vault. **Triple backup 3-2-1** cada noche 02:00 con 2 artefactos cifrados solo con su clave en 3 sitios. **Updates**

1-click como tú con semáforo y backup previo.

■■ Cloudflare R2 — por qué 10 GB gratis

Tus backups usan **R2** (`src/lib/storage.ts:3` + `storage-provider.ts:11`). R2 da **10 GB gratis + 0 € egress** (vs S3 que cobra por descarga) — 2-10 MB/noche × 90 días = 0,2-0,9 GB → gratis. Por eso `hosting_and_business_strategy.md:113` elige R2. Crear cuenta: `dash.cloudflare.com/sign-up` → **R2** → **Enable** → **Create bucket cliente-backups / palmerp-vault** → **Manage R2 API Tokens** → **Access Key/Secret** (ver cap. 5.3b).

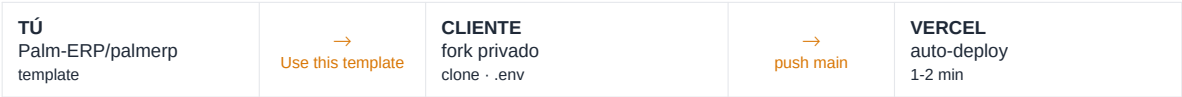
■ En una frase

Es tu ERP, pero cada cliente duerme en **su casa**, con **3 llaves** y **la misma herramienta que tú** para actualizar.

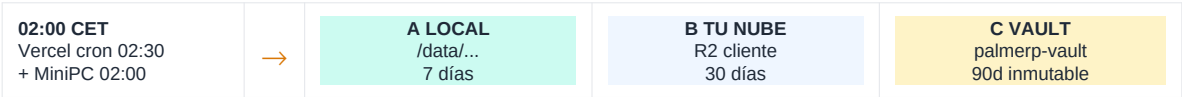
2 Mapa visual — cómo encaja todo

Fleet — una casa por cliente

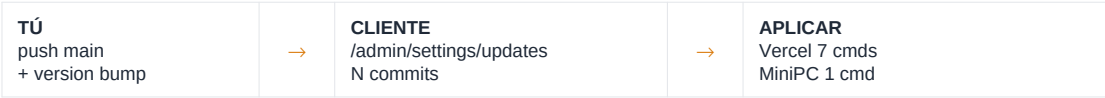
Tu repo base es `template`. Cliente hace `fork privado` → `clone` → `.env` → `provision` → **Vercel import**. Tu base es **upstream: git remote add upstream https://github.com/Palm-ERP/palmerp.git**. Cuando tú haces `git push origin main` en palmerp, cada cliente hace `git fetch upstream && git merge upstream/main && git push origin main` y su Vercel despliega. El `control.palmerp.es` solo ve `heartbeats` (no sus datos).



Backup 02:00 — 3 destinos



Update 1-click



3 Por qué cada decisión (sin humo)

Decisión	Por qué	Alternativa descartada
Fleet 1 DB/cliente	Aislamiento real, GDPR limpio, se puede ir con <code>pg_dump</code>	Single-DB: barato pero 1 bug filtra datos, backup solo JSON
Vercel por cliente	Él paga su factura/dominio/env, tú no eres cuello	Wildcard 1 Vercel: 1 caída para todos
Pooler 6543 ?pgbouncer	Serverless necesita pooler	Direct 5432: too many connections
3-2-1 triple backup	3 copias, 2 medios, 1 off-site	Solo R2: si borras bucket adiós
A LOCAL MiniPC	No depende de internet	Solo nube: si Vercel cae a 02:00 no hay backup
B R2 cliente	Él controla su 2ª copia	Solo tu vault: si tú caes él nada
C vault 90d immutable	Tu garantía aunque él borre B	Sin vault: borra A+B → nada
Solo clave cliente	Tú guardas <code>.enc</code> opaco, GDPR ok	Clave compartida: tú ves sus datos
Logical + Physical	Ligero vs clon exacto	Solo uno: secuencias o peso
02:00 CET	Valle, menos locks	23:30: más carga
MiniPC siempre ON	Garantiza A+physical	Apagable: noche sin A
Updates como tú	1 flujo, 1 verdad	Deploy central: riesgoso
Backup antes update	Rollback	Sin backup: migration rota sin vuelta
Build antes push	No despliegas roto	Push directo: Vercel falla en prod

4 Manual de Usuario — lo que ves cada día

Si eres dueño, lee solo este capítulo. No toques .env ni MiniPC.

Entrar

1) Abre <https://tu-dominio.vercel.app/login> 2) Email + contraseña (te la dio el instalador) 3) Entrar → /admin (dashboard, contactos, ventas...)

Tu minuto cada mañana (café en mano)

No abras 3 consolas. Mira un semáforo en /admin/settings/updates o /api/fleet/backup-heartbeat:

Color	Significa	Qué hacer
Verde	Backup <24h en 3 sitios, al día	Nada. Trabaja.
Amarillo	24-48h o 2 destinos	Pide backup manual al instalador
Rojo	>48h o fallido	Llama hoy

Si borras algo

Pide restore **lógico** de anoche (no borra, solo añade lo que falta): instalador hace **GET /api/admin/restore?tenantId=xxx** → **elige *.logical.json.gz.enc** → **descarga del MiniPC/R2/vault** → **POST dryRun:true** → **preview** → **dryRun:false**. Si hay que reconstruir todo, es **físico** del MiniPC: **psql "\$DATABASE_URL" < dump.sql** en staging primero.

Pedir actualización

Ajustes → Actualizaciones → **Comprobar** → si sale **3 commits por detrás** copia el bloque **Vercel** (7 líneas) o **MiniPC** (**npm run fleet:update**) y lo ejecuta el instalador. Si el build falla, no se despliega. Siempre hay backup de esa noche.

5 Manual del Instalador — de 0 a 100% paso a paso

Tu contrato. Cada paso tiene comando literal y salida esperada. No avances si falla.

Mapa 0-100%

Pre-requisitos (5') → Fork+upstream (2') → Supabase (3') → Claves (1') → .env (3') → npm ci+prisma+provision (4') → Vercel (3') → MiniPC (5') → Verificación (4') → Handover (2')

5.1 Pre-requisitos

- GitHub privado cliente ■ Vercel Team cliente ■ Supabase eu-central-1 ■ R2 cliente (Account ID+bucket+keys) B 30d ■ R2 PalmerP palmerp-vault C 90d ■ MiniPC Ubuntu 22/24 Node≥20.9 50GB siempre ON SSH ■ Datos slug/name/adminName/adminEmail/domain ■ Tu portátil: node≥20.9 npm git openssl curl jq

```
node -v • npm -v • git --version • openssl version
```

5.2 GitHub — fork + upstream

- 1

Template
https://github.com/Palm-ERP/palmerp → Use this template → cliente-x Private sin Include all branches
- 2

Si no es template
git clone https://github.com/Palm-ERP/palmerp.git cliente-x && cd cliente-x && git remote remove origin && gh repo create cliente-x --private --source=. --remote=origin --push
- 3

Upstream siempre
git remote add upstream https://github.com/Palm-ERP/palmerp.git && git remote -v # origin+upstream

Cuando publiques mejora: **git fetch upstream && git merge upstream/main --no-edit && git push origin main** → Vercel cliente despliega

5.3 Supabase — pooler 6543

Supabase → New Project cliente-x eu-central-1 → espera 2' → Project Settings → Database → Connection string → URI → **Pooler 6543 ?pgbouncer=true** → copia **DATABASE_URL**. Fleet = 1 DB por cliente.

```
postgres://postgres:xxxxx:[PASSWORD]@aws-0-eu-central-1.pooler.supabase.com:6543/postgres?pgbouncer=true
```

```
psql "postgresql://..." -c "select 1" # → 1
```

5.4 Claves — una vez

```
openssl rand -base64 32 | tr -d '\n' # BACKUP_ENCRYPTION_KEY 44 chars SOLO cliente QR caja fuerte
openssl rand -base64 32 | tr -d '\n' # NEXTAUTH_SECRET
openssl rand -base64 32 | tr -d '\n' # CRON_SECRET
openssl rand -base64 32 | tr -d '\n' # FLEET_API_KEY
```

■ ■ No a Git

.gitignore ignora .env*. Verifica git status que no aparece.

5.5 .env — todas las vars

cd cliente-x && cp .env.example .env.local && nano .env.local → rellena mínimo y replica idéntico en Vercel → Settings → Environment Variables (Production)

Var	Ejemplo	Si falta
DATABASE_URL	pooler 6543	migrate/backup fallan
NEXTAUTH_*	URL + secret	login loop
BACKUP_ENCRYPTION_KEY	44 chars	gz sin cifrar, auth failed
BACKUP_LOCAL_DIR	/data/backups/palmerp	usa default
CLIENT_BACKUP_S3_*	R2 cliente B	B not configured PARTIAL
PALMERP_VAULT_R2_*	vault PalmerP C	C not configured
CRON_SECRET	Bearer	401 cron
FLEET_API_KEY	Bearer	heartbeat no envía
PALMERP_UPSTREAM_*	Palm-ERP/palmerp main	check usa default

5.6 Instalar y provisionar

```
npm ci --legacy-peer-deps # added packages
npx prisma generate # ✓ Generated
npx prisma migrate deploy # 3 applied
npm run instance:provision -- --slug cliente-x --name "Cliente X S.L." --domain cliente-x.palmerp.es --admin-name "Ana" --admin-email ana@cliente.es
--admin-password "Cambia123!" # Tenant seeded
# atajo: node scripts/template-bootstrap.mjs (genera .env+claves+provisiona)
npx tsc --noEmit --skipLibCheck # sin output = OK
npm run build # ✓ 67 rutas
./init.sh # [OK]
npm run dev # http://localhost:3000
```

5.7 Vercel — import + auto-deploy

vercel.com → Add New Project → Import cliente-x → Build prisma generate && next build → Add Env (las de 5.5 Production) → Deploy → Visit → Domains → Add cliente.es → Cron Jobs verifica 30 2 * * * /api/cron/daily-backups → git commit --allow-empty -m "test" && git push origin main → Deployments verde

5.8 MiniPC — agente 02:00

```
ssh mini@192.168.1.50
sudo apt update && sudo apt install -y nodejs npm postgresql-client git curl jq && node -v && pg_dump --version
sudo mkdir -p /data/backups/palmerp && sudo chown $USER:$USER /data/backups/palmerp
git clone https://github.com/cliente/cliente-x.git /opt/palmerp && cd /opt/palmerp && npm ci --legacy-peer-deps && cp .env.example .env && nano .env #
MISMAS vars Vercel
npm run backup:agent # o node scripts/backup-agent.mjs --once → 2 .enc + B/C uploaded
ls -lh /data/backups/palmerp/tenants/*/daily/
sudo cp scripts/systemd/palmerp-backup.* /etc/systemd/system/ && sudo systemctl daemon-reload && sudo systemctl enable --now palmerp-backup.timer
&& systemctl list-timers | grep palmerp
sudo systemctl start palmerp-backup.service && journalctl -u palmerp-backup.service -n 50 --no-pager
```

5.9 Verificación — no entregues sin esto

```
npx prisma generate && npx tsc --noEmit --skipLibCheck && npm run build && ./init.sh # todo [OK]
curl -H "Authorization: Bearer $CRON_SECRET" https://cliente-x.vercel.app/api/cron/daily-backups | jq .summary # success 1
npm run backup:agent && ls -lh /data/.../daily/ # 2 .enc
curl -H "Bearer $FLEET_API_KEY" https://control.palmerp.es/api/fleet/backup-heartbeat | jq .status # green <24h
npm run fleet:check # Al día / N commits
curl https://cliente-x.vercel.app/api/admin/updates | jq .reason
open https://cliente-x.vercel.app/admin/settings/updates
open https://cliente-x.vercel.app/login # ana@cliente.es
```

6 Triple Backup 3-2-1 — tus 3 seguros

Qué guarda: **Logical JSON** (Tenant/Users/Contacts/Shops/Orders/AuditLogs filtrado tenantId → gzip→AES-GCM→sha256→BackupLog) y **Physical pg_dump** (clon exacto solo MiniPC → gzip→encrypt). Dónde: **A LOCAL 7d** /data, **B R2 cliente 30d** CLIENT_BACKUP_S3_*, **C Vault palmerp-vault 90d immutable** (vault/tenants/... sin delete). Código: src/lib/backup/crypto.ts, storage-provider.ts (3 providers), runner.ts (retención), daily-backups 02:30, backup-agent.mjs.

■ Tranquilidad

Aunque Vercel+Supabase caigan, quedan 2 copias. Vault solo lo abre su clave.

7 MiniPC — copia A y cerebro local

Siempre ON. Dos roles: 1) Guardián backup físico (único que crea physical) 2) Cerebro local Ollama + vector DB que aprende de su negocio sin enviar datos fuera (nightly ETL → embeddings → /api/insights). Sin MiniPC esa noche no hay A ni physical (amarillo).

8 Actualizar como yo — 1 click, sin miedo

Tú publicas: **package.json version 0.1.0→0.2.0 + git push origin main en Palm-ERP/palmerp**. Cliente ve en **/admin/settings/updates** via **GET /api/admin/updates → src/lib/fleet/upstream.ts** (GitHub API o git ls-remote, git rev-list --count, semver) → **N commits por detrás + backup age**.

Vercel (7 cmds copia/pega):

```
git fetch upstream
git merge upstream/main --no-edit --no-ff
npm ci --legacy-peer-deps
npx prisma generate
npx prisma migrate deploy
npm run build
git push origin main # Vercel 1-2 min
```

MiniPC (1 cmd):

```
npm run fleet:update # = --apply → backup → fetch → merge → npm ci → migrate → build → push → pm2 restart
npm run fleet:check # solo informa
node scripts/fleet-update.mjs --dry-run # simula
No push si build falla; merge --abort si conflicto; GITHUB_TOKEN si repo privado; PALMERP_UPSTREAM_REPO en .env
```

APIs: /api/admin/updates, /api/fleet/check-update, lib/fleet/version.ts. Seguridad: build antes push, backup previo 3 destinos.

9 Operación diaria — 1 minuto

1) /admin/settings/updates o curl /api/fleet/backup-heartbeat | jq .status → verde? sigue 2) Amarillo 24-48h → curl cron o backup:agent 3) Rojo >48h → systemctl status palmerp-backup.timer, pg_dump --version, env grep BACKUP, journalctl

Logs: BackupLog (destination/fileKey/checksum/size/encrypted/status) + AuditLog BACKUP_RUN/HEARTBEAT. PARTIAL = algún destino falló pero al menos uno guardó.

10 Seguridad y claves

BACKUP_ENCRYPTION_KEY 44 chars: solo cliente, vault .enc opaco, pierdes clave → auth failed forever. Guarda en Vercel Env + MiniPC .env + QR papel caja fuerte + NAS. NEXTAUTH_SECRET/CRON_SECRET/FLEET_API_KEY: openssl rand -base64 32. .env* nunca a Git. RLS tenant_id + where + withTenantContext. Vault 90d immutable sin delete.

11 Troubleshooting

Error	Causa	Fix
Can't find DATABASE_URL	.env no cargado	cp .env.example .env.local
P1001 Can't reach	pooler 5432 sin ?pgbouncer	6543 ?pgbouncer=true
Ya existe slug	Tenant existe	prisma studio borra
401 cron	CRON_SECRET mal	Vercel Env = Bearer
not configured B/C	R2 vars vacías	rellena Vercel+MiniPC
Solo LOGICAL Vercel	normal	physical solo MiniPC
BACKUP_KEY no configurada	.env MiniPC vacío	copia de Vercel
pg_dump not available	no instalado	apt install postgresql-client
auth failed restore	clave distinta	usa clave que cifró
git push no despliega	no conectado	Import fork branch main
fleet always 1 behind	sin fetch	git fetch upstream

```
vercel logs --follow
journalctl -u palmerp-backup.service -n 100
ls -lh /data/backups/palmerp/tenants/*/daily/
npx prisma studio
```

12 Checklists y entrega

Instalador — firma antes de irte

- Clave en 4 sitios (Vercel, MiniPC, QR, NAS)
- DATABASE_URL pooler psql select 1 OK
- prisma generate+migrate OK
- build 67 rutas + init.sh OK + tsc OK
- Vercel import main auto-deploy push OK
- Cron 02:30 visible
- MiniPC /data ok pg_dump OK 2 .enc B/C OK
- systemd timer 02:00 enable --now OK
- curl cron success 3 destinos
- heartbeat verde <24h
- restore dryRun:true OK
- upstream git remote -v origin+upstream
- updates /admin/settings/updates verde + fleet:check OK
- Handover PDF+QR+accesos
- Cliente sabe push/fetch/semáforo/updates como tú

Entrega dueño: Manual PDF + QR clave en sobre + invites Vercel/Supabase/GitHub ADMIN + URLs <https://cliente-x.vercel.app> + /admin + control.palmerp.es + frase: “Si ves verde y guardas la clave en caja fuerte, estás cubierto aunque falle Vercel. Yo veo el vault pero no puedo abrirlo sin ti. Actualizas como yo en /admin/settings/updates.”

13 Glosario y referencias

Término	Qué es
Fleet	1 casa por cliente vs Single-DB compartida
Upstream	Palm-ERP/palmerp template base
Logical JSON	JSON tenantId → gzip→encrypt, Vercel+MiniPC
Physical	pg_dump --no-owner → gzip→encrypt, solo MiniPC
Vault immutable	palmerp-vault 90d sin delete cliente
Fleet update	fetch→merge→npm ci→migrate→build→push

Referencias archivo:línea: package.json:1, vercel.json:1, prisma/schema.prisma:1, src/lib/db.ts:1, src/lib/backup/*:1, src/lib/fleet/*:1, src/app/api/cron/daily-backups, src/app/api/admin/updates, scripts/*:1, docs/TEMPLATE-INSTALADOR.md:1, docs/architecture.md

PalmerP — Hecho con mimo en Europa — 2026 — Si este manual te dio tranquilidad, cumplió su misión. ■